

NHÓM 10

Resource Design & Data Modeling

Nội dung chính

01

Thiết kế Resource Tree

Nguyên tắc thiết kế cấu trúc tài nguyên phân cấp và quản lý relationships

02

Chiến lược Phân trang

Offset/Limit, Cursor-based, Page-based pagination

03

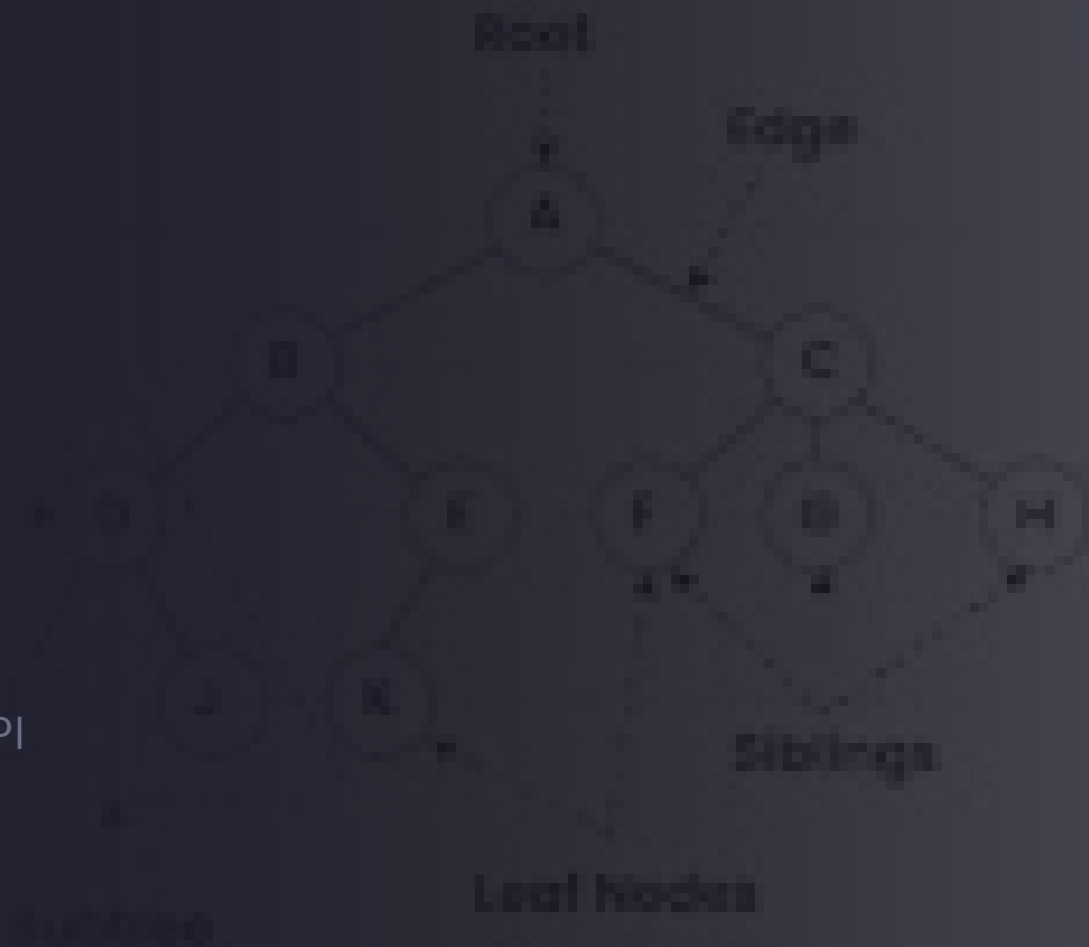
So sánh và Lựa chọn

Phân tích ưu/nhược điểm để lựa chọn chiến lược phù hợp

01

Thiết kế Resource Tree

Nguyên tắc thiết kế cấu trúc tài nguyên phân cấp trong REST API



Nguyên tắc cơ bản của Resource Modeling



1. Xác định Resource

Xác định các entities chính trong hệ thống. Ví dụ: `users`, `orders`, `products`, `comments`



2. Định nghĩa Relationships

Thiết lập mối quan hệ giữa các resources: `users` có `orders`, `posts` có `comments`



3. Thiết kế URIs

Tạo URIs có ý nghĩa, nhất quán phản ánh phân cấp và quan hệ giữa các resources

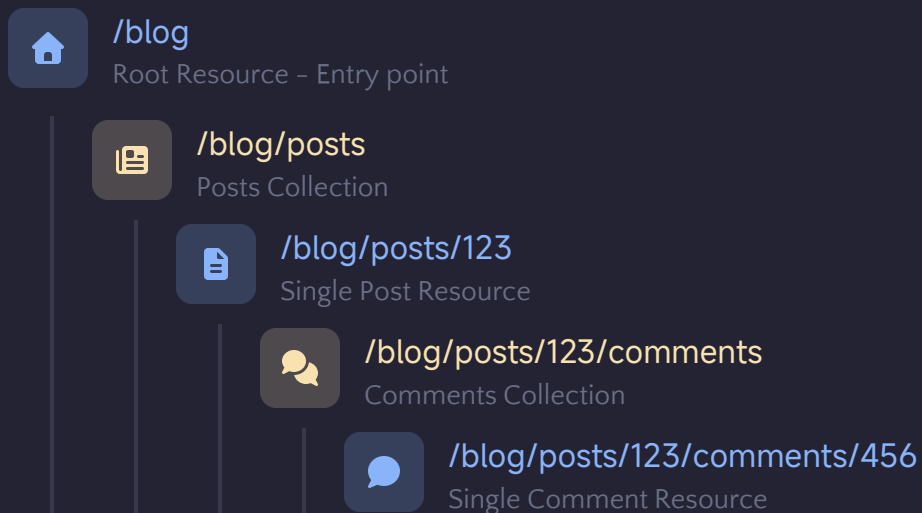


4. Biểu diễn Resources

Chọn định dạng phù hợp (thường là JSON) để biểu diễn resource data với đầy đủ fields và metadata

Resource Tree

Ví dụ: Blog API



Mối quan hệ Cha-Con

- Blog chứa Posts
- Post chứa Comments
- URL path thể hiện rõ hierarchy

Lợi ích

- ✓ Dễ hiểu
- ✓ Dễ đoán
- ✓ Self-documenting API

Quản lý quan hệ giữa các resource



Resource Embedding

Nhúng resources trực tiếp vào parent resource response

```
GET /posts/123

{
  "id": 123,
  "title": "REST API Guide",
  "content": "...",
  "comments": [
    { "id": 1, "text": "Great!" },
    { "id": 2, "text": "Thanks!" }
  ]
}
```

- + **Ưu điểm:** Giảm số requests, data đầy đủ trong một call
- **Nhược điểm:** Response lớn, over-fetching, khó cache



Resource Identifiers

Sử dụng unique identifiers để thể hiện relationships

```
GET /posts/123

{
  "id": 123,
  "title": "REST API Guide",
  "content": "...",
  "comments": [1, 2, 3],
  "author_id": 456
}
```

- + **Ưu điểm:** Response ngắn, dễ cache từng resource
- **Nhược điểm:** Nhiều requests hơn để lấy đầy đủ data

02

Chiến lược Phân trang

Các phương pháp phân trang dữ liệu trong REST API

Tại sao cần Pagination?



Performance

Load toàn bộ dataset có thể làm quá tải servers và client applications, dẫn đến response chậm

Vấn đề:

- Query chậm
- Memory overhead lớn
- Network bandwidth cao



Usability

Users khó điều hướng và hiểu dataset lớn khi tất cả hiện thị cùng 1 lúc

Giải pháp:

- Chia nhỏ thành các phần
- Dễ dàng điều hướng (next/prev/jump)



Security

Giới hạn data trong mỗi API call giúp bảo vệ thông tin nhạy cảm

Lợi ích:

- Giảm khả năng bị tấn công
- Rate limiting hiệu quả hơn



Những trường hợp cần Pagination

- ✓ Social media feeds
- ✓ E-commerce catalogs
- ✓ Transaction lists
- ✓ Search results

Offset-Based Pagination

Cơ chế hoạt động

Sử dụng **offset** (vị trí bắt đầu) và **limit** (số lượng records) để xác định data cần retrieve

Request:

```
GET /items?offset=10&limit=10
```

Lấy 10 items bắt đầu từ vị trí thứ 10

SQL Query:

```
SELECT * FROM items
ORDER BY id
LIMIT 10 OFFSET 10
```



Tính toán Offset

$$\text{offset} = (\text{page_number} - 1) \times \text{page_size}$$

Ví dụ: Page 3 với page size 10 → offset = 20

Response Format

```
{
  "data": [...],
  "meta": { "total": 1000, "limit": 10, "offset": 10 },
  "links": {
    "self": "/items?offset=10&limit=10",
    "next": "/items?offset=20&limit=10",
    "prev": "/items?offset=0&limit=10",
    "first": "/items?offset=0&limit=10",
    "last": "/items?offset=990&limit=10"
  }
}
```

Ưu điểm & Nhược điểm

+ Ưu điểm

- Đơn giản, dễ implement
- Native support trong SQL
- Có thể jump đến bất kỳ page nào

- Nhược điểm

- Performance giảm khi offset lớn
- Data drift khi có thay đổi
- COUNT(*) query chi phí cao

Page-Based Pagination

Cơ chế hoạt động

Sử dụng **page** parameter để chỉ định số thứ tự trang muốn lấy

Request:

```
GET /items?page=2&limit=20
```

Lấy page thứ 2, mỗi page 20 items

Công thức tính offset:

$\text{offset} = (\text{page} - 1) \times \text{limit}$

Ví dụ: Page 2, limit 20 $\rightarrow$ offset = 20

Response với Page Metadata

```
{ "data": [...], "pagination": { "current_page": 2,
"per_page": 20, "total_pages": 50, "total_items": 1000 },
"links": { "first": "/items?page=1&limit=20", "prev":
"/items?page=1&limit=20", "next": "/items?page=3&limit=20",
"last": "/items?page=50&limit=20" } }
```

UI/UX Friendly



Thao tác quen thuộc với users

So sánh với Offset-Based

== **Giống:** Cả hai đều dựa trên offset/limit SQL

≠ **Khác:** Page-based dễ hiểu hơn cho end users

⚠ **Lưu ý:** Vẫn có vấn đề hiệu năng và data drift như offset-based

Cursor-Based Pagination

Cơ chế hoạt động

Sử dụng **cursor** (unique identifier/token) để đánh dấu vị trí trong dataset

Request:

```
GET /items?cursor=abc123&limit=10
```

Cursor là ID của last item từ page trước

SQL Query:

```
SELECT * FROM items
WHERE id > 'abc123'
ORDER BY id ASC
LIMIT 10
```

</> Cursor Encoding

Cursor thường được base64 encode để obfuscate:

dXNlcjpxMDdRQ1JQQTQ= → user:W07QCRPA4

Giúp API thay đổi logic mà không break clients

Response Format

```
{ "data": [...], "next_cursor": "xyz789", "limit": 10, "links": {
  "self": "/items?cursor=abc123&limit=10", "next":
  "/items?cursor=xyz789&limit=10", "prev":
  "/items?cursor=prevCursor&limit=10", "first":
  "/items?cursor=firstCursor&limit=10" } }
```

Ưu điểm & Nhược điểm

+ Ưu điểm

- Performance O(1) – ổn định ở mọi page
- Không bị data drift
- Không cần COUNT query
- Hoàn hảo cho infinite scroll

- Nhược điểm

- Không thể jump đến page bất kỳ
- Complex hơn để implement
- Cursor có thể expire

03

So sánh và Lựa chọn

Phân tích ưu/nhược điểm và hướng dẫn lựa chọn chiến lược phù hợp



So sánh hiệu năng: Offset vs Cursor



Offset-Based: $O(N)$

Database phải **scan và discard** tất cả rows từ đầu đến offset

Ví dụ: OFFSET 10000

1. Scan 10,000 rows
2. Discard 10,000 rows
3. Return 10 rows

- ❌ Page 1: -10ms
- ❌ Page 1000: -3000ms
- ⚠️ Chậm dần theo offset



COUNT(*) Query

Offset-based thường cần COUNT(*) để tính total pages → **expensive trên large tables**



Cursor-Based: $O(1)$

Database dùng **index** để jump trực tiếp đến cursor record

Ví dụ: cursor=xyz789

1. Index lookup: $O(\log N)$
2. Jump to cursor position
3. Return next 10 rows

- ✅ Page 1: -10ms
- ✅ Page 10000: -10ms
- ✅ Performance ổn định

Vấn đề Data Drift



Offset-Based: Data Drift

Khi data thay đổi (thêm/xóa) giữa các requests, user có thể thấy **duplicate** hoặc **miss records**

Ví dụ:

Request 1: GET /items?offset=0&limit=5

→ Items: [A, B, C, D, E]

⚠ New item X inserted at top

Request 2: GET /items?offset=5&limit=5

→ Items: [E, F, G, H, I]

✗ Item E bị duplicate!



Cursor-Based: Consistent

Cursor trở đến **specific record**, không phải position → data luôn consistent

Ví dụ:

Request 1: GET /items?limit=5

→ Items: [A, B, C, D, E], cursor=E

⚠ New item X inserted at top

Request 2: GET /items?cursor=E&limit=5

→ Items: [F, G, H, I, J]

✅ Không duplicate, không miss!



Khi nào consistency quan trọng?

- Financial transactions
- Real-time activity feeds
- Chat/message histories
- Audit logs

Khi nào dùng Offset-Based Pagination?



1. Dataset nhỏ

Dưới **10,000 rows** – performance vẫn tốt, không cần optimize phức tạp



2. UI cần "Jump to Page"

Admin dashboards, legal archives – users cần navigate đến page cụ thể



3. Data tĩnh, ít thay đổi

Historical records, reference data – **data drift** không phải vấn đề



4. Prototypes & Internal Tools

Speed of development quan trọng hơn scalability – **nhANH, đơn giản**



5. Search Results

Users **hiếm khi đi quá page đầu** – offset nhỏ nên performance vẫn tốt



Rule of Thumb

Offset works fine cho **small, rarely changing datasets**. Khi data lớn hoặc thay đổi liên tục → chuyển sang Cursor.

Khi nào dùng Cursor-Based Pagination?



1. Dataset lớn

Hàng triệu rows – cursor giữ performance ổn định ở mọi page depth



2. Infinite Scroll / Load More

UI pattern phổ biến trên **mobile apps**, **social feeds** – không cần jump to page



3. Data thay đổi liên tục

Real-time feeds, **activity streams** – cần consistency, không cho phép duplicate/miss

Real-world Examples



Twitter API

Timeline feeds, tweets



Instagram Graph API

Posts, comments, likes



Slack API

Messages, channels



Facebook API

News feed, notifications



Industry Standard

Cursor-based là **de facto standard** cho social media feeds và real-time applications.

Query Parameters bổ sung cho Pagination



Filtering

Lọc data trước khi phân trang

```
?status=active  
?category=technology  
?date_from=2024-01-01  
?date_to=2024-12-31
```

Ví dụ:
`/posts?category=tech&page=1`



Sorting

Sắp xếp data trước khi phân trang

```
?sort=created_at  
?sort=-created_at  
?sort=name&order=asc  
?sort=-priority,created_at
```

Ví dụ:
`/posts?sort=-created_at&limit=10`



Search

Tìm kiếm text trong data

```
?q=keyword  
?search=rest+api  
?query=pagination
```

Ví dụ:
`/posts?q=rest+api&page=1`

Kết hợp tất cả parameters

```
GET /api/posts?category=technology&status=published&sort=-created_at&q=rest+api&cursor=abc123&limit=20
```



Filter: category, status



Sort: -created_at



Search: rest api



Paginate: cursor, limit